

A novel approximate multiplier based on improved logarithmic and antilogarithmic conversions

Gennaro Di Meo
DIETI
University of Naples
Federico II
Napoli, Italy
gennaro.dimeo@unina.it

Davide De Caro
DIETI
University of Naples
Federico II
Napoli, Italy
dadecaro@unina.it

Luca Tegazzini
DIETI
University of Naples
Federico II
Napoli, Italy
luca.tegazzini@unina.it

Ettore Napoli
DIEM
University of Salerno
Fisciano, Italy
enapoli@unisa.it

Antonio G. M. Strollo
DIETI
University of Naples
Federico II
Napoli, Italy
astrollo@unina.it

Abstract—In this paper, we propose a novel approximate multiplier with error-improved logarithmic and antilogarithmic conversion. In logarithmic multiplication, the forward and backward conversions to the logarithm domain introduce errors in the product. To recover accuracy, we apply an offset when logarithm and antilogarithm are computed, and find suitable values for these offsets in order to make the approximation error with zero mean. The hardware structure of the proposed multiplier is described for signed arithmetic, and a novel hardware-efficient approach, able to compute the sign of the output, is also proposed. Error metrics and synthesis analyses in a 28nm CMOS technology show that our multiplier offers the best accuracy-hardware performance for $MRED < 4 \times 10^{-2}$. Furthermore, remarkable results are achieved also in JPEG compression, exhibiting SSIM and PSNR metrics competitive with the state-of-the-art.

Keywords—Approximate computing, logarithmic multiplier, logarithmic and antilogarithmic conversion, error correction

I. INTRODUCTION

In the last few decades, Approximate Computing has emerged as a promising design strategy for the realization of a large variety of digital systems, ranging from Fast Fourier Transform processors to clock generators [1]–[2], providing to exploit the error-resilient nature of these applications to improve their hardware complexity [3],[4]. Binary multipliers, which are central circuits to the realization of digital signal processing algorithms, are responsible for large power consumption and area occupation. Accordingly, the adoption of approximation based design strategies for binary multipliers constitutes a meaningful solution to achieve hardware improvements with acceptable accuracy loss.

In literature, several techniques have been proposed to design signed and unsigned approximate multipliers, covering approximate compression, column pruning, or input segmentation [5]–[7]. Among these, multiplication in the logarithmic number system (LNS) occupies an interesting position, providing to compute the product exploiting addition [8]. In this case, three steps allow to find the result: (i) logarithm conversion of the inputs, (ii) addition of the logarithms, (iii) antilogarithm computation of the sum. Several efforts have been focused to realize hardware efficient logarithmic multipliers while improving the accuracy of computation. In [9],[10] a correction term is added to compensate the approximation error, and operand truncation is also employed to lower circuitual complexity. Additionally, [10] extends the multiplication to the signed arithmetic involving XOR gates. The work [11] employs approximate adders to reduce power and area, while [12]

exploits static segmentation before logarithm computation.

In this paper, we propose a novel logarithmic multiplier with logarithmic and antilogarithmic conversions offering low approximation error. For the sake of generality, we focus the attention on signed arithmetic. In our design, an offset is introduced in the computation of both the logarithm and the antilogarithm. Then, we find proper values for these offsets in order to have conversions with zero mean error. Only two adders are involved in the design, one to correct the logarithms, and one to correct the antilogarithm. In addition, we also propose a novel architecture able to alleviate the complexity of the sign and zero computation circuitry. Accuracy assessment reveals error metrics comparable to the state-of-the-art, while synthesis results in TSMC 28nm CMOS technology show higher performance both in terms of power-delay product (PDP) and area-delay product (ADP). The proposed circuit is also competitive in JPEG compression, achieving state-of-the-art performances.

The paper is organized as follows: section II introduces the multiplication in the LNS; section III describes the proposed correction method and the implementation details of the multiplier. Error metrics and hardware results are in section IV, while section V discusses JPEG compression performance. Section VI concludes the paper.

II. LOGARITHMIC MULTIPLICATION

The multiplication in the LNS finds the product $P=A \cdot B$ between two unsigned numbers exploiting the summation

$$\log_2(P) = \log_2(A \cdot B) = \log_2(A) + \log_2(B) \quad (1)$$

Let express the n -bit number A as follows:

$$A = \sum_{i=0}^{n-1} a_i 2^i = 2^{ka} \cdot (1 + x_A) \quad (2)$$

where a_i is the i -th bit of A , ka is an integer number pointing to the position of the leading one in A , and x_A is defined as

$$x_A = \sum_{i=-ka}^{-1} a_i 2^i \quad (3)$$

The mantissa x_A gathers the bits at the right of the leading one and lays in the range $[0, 1)$. Similar representation also holds for B . Starting from (2), the base 2 logarithm of A is

$$\log_2(A) = ka + \log_2(1 + x_A) \quad (4)$$

Estimating $\log_2(1+x_A)$ is not straightforward and requires intensive calculations. In order to moderate the complexity, the following approximation is proposed by Mitchell in [8]:

$$\log_2(1 + x_A) \approx x_A \quad (5)$$

Then, the logarithm in (4) and the sum in (1) become

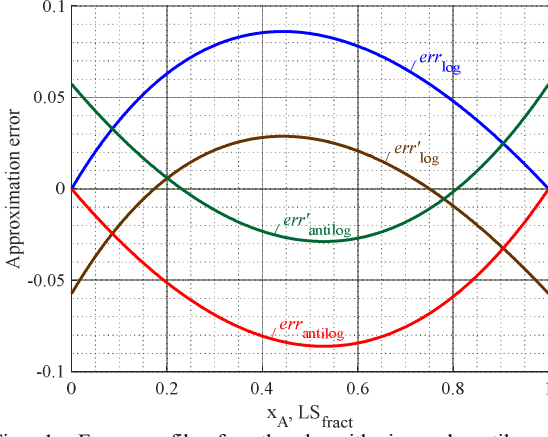


Fig. 1. Error profile for the logarithmic and antilogarithmic approximation in the Mitchell [8] and in the proposed multiplier.

$$\log_2(A) \approx ka + x_A \quad (6)$$

$$\log_2(A \cdot B) \approx ka + x_A + kb + x_B \quad (7)$$

where kb and x_B are the leading one pointer and the mantissa of B . Once (7) is computed, the antilogarithm operation allows to get the approximate value of P . Naming LS the sum $ka + x_A + kb + x_B$, the approximate product P' is computed as [8]

$$P' = 2^{LS} \approx \begin{cases} 2^{ka+kb} \cdot (1 + x_A + x_B) & \text{if } x_A + x_B < 1 \\ 2^{ka+kb+1} \cdot (x_A + x_B) & \text{if } x_A + x_B \geq 1 \end{cases} \quad (8)$$

Please note that the above algorithm can be extended also to signed arithmetic just performing multiplication between the absolute value of the inputs [10],[12]. Finally, the sign is recovered by inverting the result if the product is negative.

III. PROPOSED TECHNIQUE

A. Theoretical approach

Before introducing the proposed correction method, let us observe that $LS = ka + x_A + kb + x_B$ has an integer and a fractional part, named LS_{int} and LS_{fract} , respectively, with LS_{fract} that is in the range $[0, 1)$. Then, (8) can be rewritten as

$$P' = 2^{(LS_{\text{int}} + LS_{\text{fract}})} = 2^{LS_{\text{int}}} \cdot 2^{LS_{\text{fract}}} \quad (9)$$

with

$$2^{LS_{\text{int}}} = \begin{cases} 2^{ka+kb} & \text{if } x_A + x_B < 1 \\ 2^{ka+kb+1} & \text{if } x_A + x_B \geq 1 \end{cases} \quad (10)$$

and $2^{LS_{\text{fract}}}$ approximated as follows:

$$2^{LS_{\text{fract}}} \approx 1 + LS_{\text{fract}} = \begin{cases} 1 + x_A + x_B & \text{if } x_A + x_B < 1 \\ x_A + x_B & \text{if } x_A + x_B \geq 1 \end{cases} \quad (11)$$

Figure 1 depicts the error profile of approximations (5), (11), defining the logarithm and the antilogarithm errors as

$$\begin{aligned} err_{\log} &= \log_2(1 + x_A) - x_A \\ err_{\text{antilog}} &= 2^{LS_{\text{fract}}} - 1 - LS_{\text{fract}} \end{aligned} \quad (12)$$

As shown, (5) underestimates the logarithm (being $err_{\log}$ always positive), whereas (11) overestimates the antilogarithm (being err_{antilog} always negative).

To improve the accuracy, we modify (5), (11) as follows:

$$\begin{aligned} \log_2(1 + x_A) &\approx x_A + c_{\log} \\ 2^{LS_{\text{fract}}} &\approx LS_{\text{fract}} + c_{\exp} \end{aligned} \quad (13)$$

where $c_{\log}$, $c_{\exp}$ are offsets to be determined. Our aim is to find $c_{\log}$, $c_{\exp}$ in order to have logarithmic and antilogarithmic conversions with zero mean error. Writing the approximation errors $err'_{\log}(x_A, c_{\log})$, $err'_{\exp}(LS_{\text{fract}}, c_{\exp})$ as follows

$$\begin{aligned} err'_{\log}(x_A, c_{\log}) &= \log_2(1 + x_A) - x_A - c_{\log} \\ err'_{\exp}(LS_{\text{fract}}, c_{\exp}) &= 2^{LS_{\text{fract}}} - LS_{\text{fract}} - c_{\exp} \end{aligned} \quad (14)$$

we compute their mean values for x_A and LS_{fract} in $[0, 1)$ and impose the zero mean condition:

$$\begin{aligned} \overline{err'_{\log}} &= \int_0^1 err'_{\log}(x_A, c_{\log}) dx_A = 0 \\ \overline{err'_{\exp}} &= \int_0^1 err'_{\exp}(LS_{\text{fract}}, c_{\exp}) dLS_{\text{fract}} = 0 \end{aligned} \quad (15)$$

The results of the integrals are shown in the following:

$$\overline{err'_{\log}} = \frac{3}{2} - \frac{1}{\ln(2)} - c_{\log}, \quad \overline{err'_{\exp}} = \frac{1}{\ln(2)} - \frac{1}{2} - c_{\exp} \quad (16)$$

Hence, solving (15), the desired coefficients are

$$c_{\log} = \frac{3}{2} - \frac{1}{\ln(2)}, \quad c_{\exp} = \frac{1}{\ln(2)} - \frac{1}{2} \quad (17)$$

As clear from Fig. 1, the errors obtained in our case have zero mean (see the brown and the green curves, varying around zero), thus allowing to improve the accuracy.

B. Hardware implementation

The block diagram of the proposed corrected logarithmic multiplier (named cLM in the following) is depicted in Fig. 2 particularized to the case of 16-bit inputs. For generality, we focused on the signed design, where A s and B s are the signed inputs and P s' is the signed output, all represented in two's complement. Fig. 2a shows the structure of the logarithmic conversion block. A bank of XOR gates allows to compute the absolute value of A s, B s performing the one's complement if their most significant bits (MSBs), s_A , s_B , are high, while a further XOR gate finds the sign s of the output. A leading one detector (LOD) stage allows to identify the

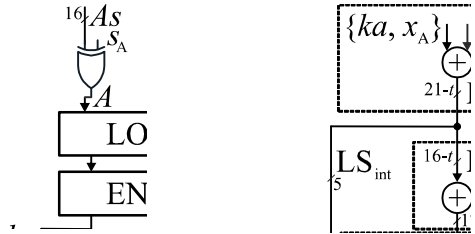


Fig. 2. (a) Logarithmic conversion block and (b) sum and antilogarithm circuitry exploiting the proposed correction technique.

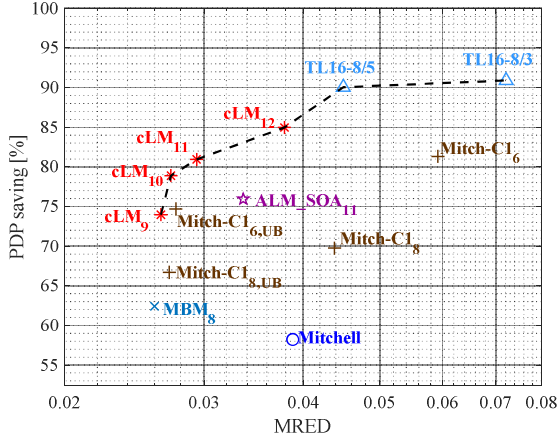


Fig. 3. PDP saving vs $MRED$. Best multipliers are on the pareto front.

leading one in the absolute values A , B , while encoders (ENC) find the pointers ka , kb . Then, starting from ka , kb , the barrel shifters extract the mantissas x_A , x_B , expressed on 15 bits. To optimize the hardware, we also truncate t least significant bits (LSBs) of x_A , x_B and append a ‘1’ at the least significant position to reduce the approximation error (see the figure). Therefore, the truncated mantissas are expressed on $16-t$ bits. It is worth noting that t parameter allows to modify the accuracy of the multiplier (as shown in sect. IV). Then, following (6), the logarithms of the inputs are obtained just concatenating ka to x_A and kb to x_B , being ka , kb and x_A , x_B integer and fractional numbers, respectively.

In Fig. 2b, the first adder computes LS exploiting the proposed correction terms for the logarithm conversion. Since $\log_2(1+x_A)$ and $\log_2(1+x_B)$ demand for the same offset $c_{\log}$, the summation (7) becomes

$$LS = ka + x_A + kb + x_B + 2 \cdot c_{\log} \quad (18)$$

that computes logarithm sum and correction with a 3-input adder simplified by one fixed input. Since $2 \cdot c_{\log} \approx 0.1146$ is a real number, a careful quantization is required to preserve multiplier accuracy. From simulation analyses, we saw that approximating $2 \cdot c_{\log}$ as ‘0.000111’ ≈ 0.1094 is sufficient to achieve satisfactory correction without degrading the precision. Once LS is computed, the integer part LS_{int} , gathering its 5 MSBs, is extracted to drive the shifter, whereas the fractional part LS_{frac} , formed by the remaining $16-t$ bits, is added to c_{exp} for the antilogarithmic conversion (see the second adder in Fig. 2b). Also in this case, we approximate $c_{\text{exp}} \approx 0.9427$, choosing the binary number ‘0.1111’ ≈ 0.9375 . Please note that this addition involves only

4 MSBs of LS_{frac} , thus having a minimum hardware impact. Finally, a XOR bank recovers the sign inverting the bits of the antilogarithm if s is high. Then, the sign bit s is appended as MSB and a multiplexer allows to set to zero the result if one of the inputs A , B is zero. Finally, the shifter applies the factor $2^{LS_{\text{int}}}$ to get the required product. We underline that [10],[12] place the sign and zero circuitry after the shifter, processing a 32-bit signal. In our case, we moved it before the shifter to process a $(17-t)$ -bit signal (see Fig. 2b), thus reducing the XOR bank and multiplexer size.

IV. PERFORMANCE ASSESSMENT

A. Accuracy analysis

Error performance are assessed in terms of normalized mean error distance ($NMED$) and mean relative error distance ($MRED$), computed by feeding the proposed multipliers with 10^7 random inputs. In addition, we compute also the probability of having relative error distance larger than 2% ($PRED$). We select implementations with truncation $t=9,10,11,12$ (corresponding to multipliers cLM_9 , cLM_{10} , cLM_{11} , cLM_{12}), able to offer an acceptable compromise between precision and hardware complexity.

Table I shows a comparison between cLM and multipliers [9], [10], [11], [12], with all circuits implemented for 16-bit signed inputs. The multiplier [9], named MBM_q , truncates q LSBs of mantissas, while [10] preserves only w bits after the barrel shifters. In addition, versions with both biased and unbiased errors are investigated (referred as $Mich-C1_w$ and $Mitch-C1_{w,UB}$). In [11], the set-one adder with 11 approximate LSBs is employed to compute the logarithm sum (the multiplier is named ALM_SOA_{11}). In [12], 8-bit segments are extracted from the inputs before logarithm conversion, and w bits are preserved after the barrel shifters. For this analysis, we consider $w=3, 5$, corresponding to multipliers $TL16-8/3$ and $TL16-8/5$. Proposed circuits offer error metrics competitive with the state-of-the-art, with $MRED$ that improves from 3.79×10^{-2} (cLM_{12}) to 2.65×10^{-2} (cLM_9). Implementations [10] exhibit results lower than cLM_9 , whereas MBM_8 [9] is the only one able to achieve better accuracy. $PRED$ and $NMED$ follow similar trends, performing between [10] and [9].

B. Hardware results

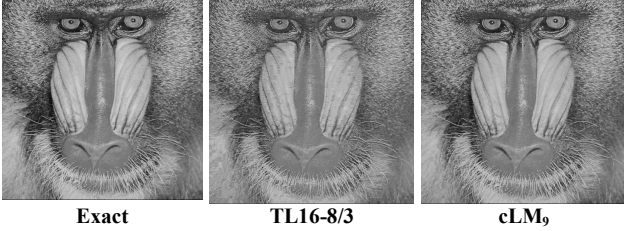
The proposed multipliers and the state-of-the-art are synthesized in TSMC 28nm technology using Cadence Genus with a clock frequency constraint of 100MHz. Area

TABLE I. COMPARISON BETWEEN THE PROPOSED MULTIPLIER AND THE STATE-OF-THE-ART IN TERMS OF ACCURACY AND HARDWARE PERFORMANCE

Multiplier	NMED	MRED	PRED	Delay [ns]	Min. delay [ns]	Area [μm^2]	Power [$\mu W@100MHz$]	ADP [$\mu m^2 \cdot ns$]	PDP [$\mu W \cdot ns$]
Exact				1.222	0.306	466.1	106.3	569.5	129.9
Mitchell [8]	9.27×10^{-3}	3.88×10^{-2}	6.52×10^{-1}	1.183	0.394	250.1	45.9	295.9 (-48.0%)	54.3 (-58.2%)
MBM ₈ [9]	6.01×10^{-3}	2.60×10^{-2}	5.34×10^{-1}	0.994	0.373	215.1	49.1	213.8 (-62.5%)	48.8 (-62.4%)
Mitch-C1 ₆ [10]	1.44×10^{-2}	5.92×10^{-2}	9.29×10^{-1}	0.862	0.341	188.1	28.1	162.2 (-71.5%)	24.3 (-81.3%)
Mitch-C1 ₈ [10]	1.06×10^{-2}	4.38×10^{-2}	7.24×10^{-1}	0.965	0.366	198.1	40.7	191.9 (-66.3%)	39.3 (-69.8%)
Mitch-C1 _{6,UB} [10]	6.83×10^{-3}	2.76×10^{-2}	6.24×10^{-1}	0.886	0.358	187.5	37.1	166.1 (-70.8%)	32.9 (-74.7%)
Mitch-C1 _{8,UB} [10]	6.72×10^{-3}	2.71×10^{-2}	6.31×10^{-1}	0.962	0.377	204.5	45.0	196.7 (-65.5%)	43.3 (-66.7%)
ALM SOA ₁₁ [11]	8.08×10^{-3}	3.36×10^{-2}	5.84×10^{-1}	0.878	0.344	184.7	35.6	162.2 (-71.5%)	31.3 (-75.9%)
TL16-8/3 [12]	1.71×10^{-2}	7.22×10^{-2}	8.26×10^{-1}	0.686	0.288	130.2	17.2	89.3 (-84.3%)	11.8 (-90.9%)
TL16-8/5 [12]	1.00×10^{-2}	4.50×10^{-2}	7.00×10^{-1}	0.682	0.312	153.3	18.9	104.6 (-81.6%)	12.9 (-90.1%)
cLM ₉	6.45×10^{-3}	2.65×10^{-2}	5.73×10^{-1}	0.871	0.360	216.0	38.9	188.1 (-67.0%)	33.8 (-74.0%)
cLM ₁₀	6.67×10^{-3}	2.72×10^{-2}	5.88×10^{-1}	0.859	0.362	204.8	31.9	175.9 (-69.1%)	27.4 (-78.9%)
cLM ₁₁	7.16×10^{-3}	2.94×10^{-2}	6.02×10^{-1}	0.842	0.350	194.4	29.4	163.7 (-71.3%)	24.8 (-80.9%)
cLM ₁₂	9.16×10^{-3}	3.79×10^{-2}	6.81×10^{-1}	0.793	0.332	184.3	24.6	146.2 (-74.3%)	19.5 (-85.0%)

TABLE II. JPEG COMPRESSION RESULTS

	$Q=50$		$Q=100$		Average	
	SSIM	PSNR [dB]	SSIM	PSNR [dB]	SSIM	PSNR [dB]
Mitchell [8]	0.950	25.6	0.950	25.6	0.950	25.6
MBM₈ [9]	0.979	34.9	0.979	34.9	0.979	34.9
Mitch-C1₆ [10]	0.932	20.4	0.932	24.0	0.932	24.0
Mitch-C1₈ [10]	0.947	25.4	0.947	25.4	0.947	25.4
Mitch-C1_{6,UB} [10]	0.978	36.2	0.978	36.1	0.978	36.2
Mitch-C1_{8,UB} [10]	0.980	35.1	0.980	35.1	0.980	35.1
ALM SOA₁₁ [11]	0.954	27.9	0.954	27.9	0.954	27.9
TL16-8/3 [12]	0.817	24.2	0.819	23.6	0.818	23.9
TL16-8/5 [12]	0.882	25.1	0.881	25.1	0.882	25.1
cLM₉	0.974	32.1	0.974	32.1	0.974	32.1
cLM₁₀	0.973	34.9	0.973	34.9	0.973	34.9
cLM₁₁	0.936	33.0	0.936	33.0	0.936	33.0
cLM₁₂	0.833	29.3	0.835	29.5	0.834	29.4

Fig. 4. JPEG compression of Mandrill with $Q=100$.

occupation and delay are obtained with post-synthesis analysis, while power consumption is estimated simulating the synthesized netlist with 10^5 random inputs, with net delays annotated in standard delay format (SDF) file, and switching activity annotated in toggle count format file (TCF). Additionally, we also report the minimum delay, obtained synthesizing the circuit with the tightest clock frequency allowing positive slack. Circuits are described in Verilog HDL, and the exact multiplier, used as reference, is implemented with the operator $*$. As shown, our proposal offers advantageous performance, achieving ADP and PDP reductions in the ranges 67%–74% and 74%–85%. Minimum delay also improves with respect to [8], [9]. Among the other circuits, only [12] offers superior results with ADP and PDP savings up to 84% and 91%. However, this performance is achieved at the cost of a visible degradation of error metrics, as demonstrated by the higher values of $NMED$, $MRED$. To put results in perspective, Fig. 3 depicts the behavior of PDP saving with respect to the $MRED$. As shown, our multipliers define the pareto front for $MRED < 4 \times 10^{-2}$ (see the black line), offering the best precision-hardware trade-off.

V. JPEG COMPRESSION

JPEG compression exploits cosine transformation to cut the high frequency content of images. This way, the storage in memory is alleviated at the cost of acceptable quality loss. Since JPEG compression involves multiplications for the forward and the backward transformation, it is suitable to assess approximate multipliers in a practical scenario. Tab. II collects the results, expressed in terms of Structural Similar Index (SSIM) and Peak Signal-to-Noise Ratio, measured in dB. Processed images are Autumn, Peppers, and Mandrill with quality factors $Q=50, 100$. For each Q , the average SSIM and PSNR are reported, as well as the overall average SSIM and PSNR is registered. Results are competitive with the state-of-the-art, offering PSNR up to 35dB with cLM₁₀ and SSIM up to 0.974 with cLM₉ on average. Fig. 4 depicts Mandrill image computed with the exact, TL16-8/3 and cLM₉ multipliers. As shown, the image for cLM₉ is very

close to the exact case, whereas TL16-8/3 exhibits visible degradations on the muzzle.

VI. CONCLUSIONS

In this paper, we proposed a novel logarithmic multiplier with reduced approximation error. To improve the accuracy, we introduced an offset in the computation of logarithm and antilogarithm, finding suitable values that allow to have conversions with zero mean error. The hardware implementation of the circuit was described, and a novel approach to compute the sign of the output was proposed. Our analyses highlighted the best accuracy-hardware performance for $MRED < 4 \times 10^{-2}$ and results comparable to the state-of-the-art in applications like JPEG compression.

REFERENCES

- [1] G. Ferreira, P. T. L. Pereira, G. Paim, E. Costa and S. Bampi, "A Power-Efficient FFT Hardware Architecture Exploiting Approximate Adders," *2021 IEEE 12th Latin America Symposium on Circuits and System (LASCAS)*, Arequipa, Peru, 2021, pp. 1-4, doi: 10.1109/LASCAS51355.2021.9667154.
- [2] D. De Caro, G. Di Meo, E. Napoli, N. Petra and A. G. M. Strollo, "A 1.45 GHz All-Digital Spread Spectrum Clock Generator in 65nm CMOS for Synchronization-Free SoC Applications," in *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 11, pp. 3839-3852, Nov. 2020, doi: 10.1109/TCSI.2020.3008481.
- [3] J. Han and M. Orshansky, "Approximate computing: An emerging paradigm for energy-efficient design," *2013 18th IEEE European Test Symposium (ETS)*, Avignon, France, 2013, pp. 1-6, doi: 10.1109/ETS.2013.6569370.
- [4] V. K. Chippa, S. T. Chakradhar, K. Roy and A. Raghunathan, "Analysis and characterization of inherent application resilience for approximate computing," *2013 50th ACM/EDAC/IEEE Design Automation Conference (DAC)*, Austin, TX, USA, 2013, pp. 1-9, doi: 10.1145/2463209.2488873.
- [5] E. Napoli, A. G. M. Strollo, E. Zacharelos and G. Di Meo, "Comprehensive Analysis of Input Order Invariant Approximate 4-2 Compressors for Binary Multipliers," *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*, Singapore, Singapore, 2024, pp. 1-5, doi: 10.1109/ISCAS58744.2024.10558503.
- [6] H. Park and E. E. Swartzlander, "Truncated Multiplication with Symmetric Correction," *2006 Fortieth Asilomar Conference on Signals, Systems and Computers*, Pacific Grove, CA, USA, 2006, pp. 931-934, doi: 10.1109/ACSSC.2006.354887.
- [7] S. Narayanamoorthy, H. A. Moghaddam, Z. Liu, T. Park and N. S. Kim, "Energy-Efficient Approximate Multiplication for Digital Signal Processing and Classification Applications," in *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 23, no. 6, pp. 1180-1184, June 2015, doi: 10.1109/TVLSI.2014.2333366.
- [8] J. N. Mitchell, "Computer Multiplication and Division Using Binary Logarithms," in *IRE Transactions on Electronic Computers*, vol. EC-11, no. 4, pp. 512-517, Aug. 1962, doi: 10.1109/TEC.1962.5219391.
- [9] H. Saadat, H. Bokhari and S. Parameswaran, "Minimally Biased Multipliers for Approximate Integer and Floating-Point Multiplication," in *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 11, pp. 2623-2635, Nov. 2018, doi: 10.1109/TCAD.2018.2857262.
- [10] M. S. Kim, A. A. D. Barrio, L. T. Oliveira, R. Hermida and N. Bagherzadeh, "Efficient Mitchell's Approximate Log Multipliers for Convolutional Neural Networks," in *IEEE Transactions on Computers*, vol. 68, no. 5, pp. 660-675, 1 May 2019, doi: 10.1109/TC.2018.2880742.
- [11] W. Liu, J. Xu, D. Wang, C. Wang, P. Montuschi and F. Lombardi, "Design and Evaluation of Approximate Logarithmic Multipliers for Low Power Error-Tolerant Applications," in *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 65, no. 9, pp. 2856-2868, Sept. 2018, doi: 10.1109/TCSI.2018.2792902.
- [12] R. Pilipović, P. Bulić and U. Lotrič, "A Two-Stage Operand Trimming Approximate Logarithmic Multiplier," in *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, no. 6, pp. 2535-2545, June 2021, doi: 10.1109/TCSI.2021.3069168.